

Colecciones

Listas, mapas, pilas, colas y conjuntos

3.1 Tipos de colecciones

Cuando necesitas guardar un número de elementos que no conoces de antemano, las colecciones son la solución: estructuras dinámicas que crecen y encogen mientras el programa se ejecuta.

Familia	Cómo se accede	Ejemplos
Listas	por posición (índice numérico)	ArrayList, LinkedList
Mapas	por clave (par clave-valor)	HashMap, TreeMap
Otras	por reglas propias de entrada/salida	pilas, colas, conjuntos

3.2 Listas — ArrayList

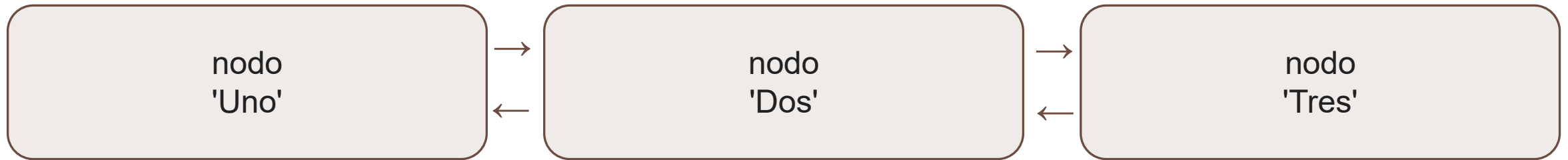
Basada en un array que crece internamente: los elementos están contiguos en memoria, uno pegado al siguiente, como las casillas numeradas de una estantería.

índice 0 'Uno'	índice 1 'Dos'	índice 2 'Tres'
-------------------	-------------------	--------------------

- Acceso por índice: rápido, $O(1)$.
- Insertar o borrar en medio de la lista: lento, $O(n)$, porque hay que desplazar el resto.

3.2 Listas — LinkedList

Lista doblemente enlazada: cada nodo guarda dos referencias, al anterior y al siguiente. Es como una cadena de personas dadas de la mano: cada una solo conoce a las de al lado.



- Acceso por índice: más lento, $O(n)$, porque hay que recorrer nodo a nodo.
- Insertar o borrar al principio o al final: rápido, $O(1)$.

3.2 Por qué cambia el rendimiento

En un `ArrayList`, `get(2)` calcula la dirección de memoria directamente (la posición de partida más 2 huecos): siempre es un único cálculo, da igual el tamaño de la lista. En una `LinkedList` no hay forma de "saltar" al nodo 2 sin pasar antes por el 0 y el 1 — de ahí el $O(n)$.

Con la inserción pasa lo simétrico: insertar en medio de un `ArrayList` desplaza una posición a todos los elementos siguientes. Insertar en una `LinkedList` no mueve ningún dato, solo cambia a qué apuntan un par de referencias — por eso es $O(1)$ en los extremos.

Si vas a acceder mucho por posición (`get(i)`), usa `ArrayList`. Si vas a insertar y borrar mucho por los extremos, `LinkedList` sale más a cuenta.

3.2 Métodos de List

```
ArrayList<String> textos = new ArrayList<>();

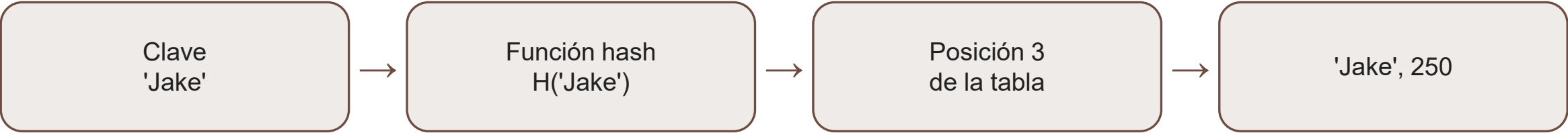
textos.add("Uno");
textos.add("Dos");
textos.add("Tres");
textos.add(2, "Dos y medio"); // se inserta entre "Dos" y "Tres"

textos.remove(1); // elimina "Dos" (posición 1)
```

Si recorres con for-each e intentas remove() dentro del bucle, Java lanza ConcurrentModificationException. Para borrar mientras recorres, usa un Iterator (o, más adelante, un stream).

3.3 Mapas — la tabla interna

Un mapa (Map) guarda pares clave-valor. HashMap calcula un número (el hash) a partir de la clave para decidir en qué posición de una tabla interna guarda cada par:

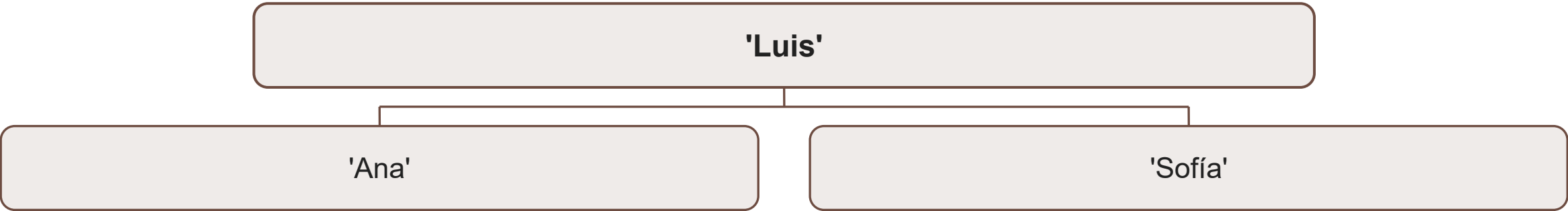


Posición	Contenido
3	'Jake' → 250
4	vacío
5	'Olivia' → 22
6	'Emma' → 242
6	'Lucía' → 19 (colisión: se encadena con la anterior)

3.3 HashMap vs TreeMap

	HashMap	TreeMap
Orden de los elementos	no garantizado	ordenados según la clave
Inserción / borrado / acceso	$O(1)$ en promedio	$O(\log n)$
Memoria	usa más memoria	usa menos, solo lo almacenado

TreeMap mantiene las claves en un árbol (rojo-negro), comparándolas entre sí. Por eso, al recorrerlo, siempre salen ordenadas de menor a mayor:



3.3 Ejemplo de HashMap

```
HashMap<String, Producto> productos = new HashMap<>();

productos.put("111A", new Producto("111A", "Monitor LG 22 pulg", 99.95f));
productos.put("222B", new Producto("222B", "Disco duro 512GB SSD", 109.95f));

productos.remove("222B");

for (String codigo : productos.keySet()) {
    System.out.println(productos.get(codigo));
}
```

HashMap usa equals()/hashCode() de la clave para calcular su posición. Con String no hay que pensar en ello, pero con una clase propia como clave tienes que sobrescribir tú los dos, o el mapa nunca la va a encontrar.

3.4 Pilas (LIFO)

Una pila sigue la regla LIFO (Last In, First Out): el último en entrar es el primero en salir, como una pila de platos.

Pila — se opera solo por la cima

'tercero' ← cima: aquí actúan push() y pop()

'segundo'

'primero'

En Java, la interfaz recomendada para pilas hoy en día es Deque (por ejemplo, ArrayDeque). La antigua clase Stack ya no se recomienda.

3.4 Pilas — métodos y ejemplo

Método	Qué hace
push(e)	añade e en la cima
pop()	quita y devuelve el elemento de la cima
peek()	consulta el elemento de la cima, sin quitarlo
isEmpty()	comprueba si la pila está vacía

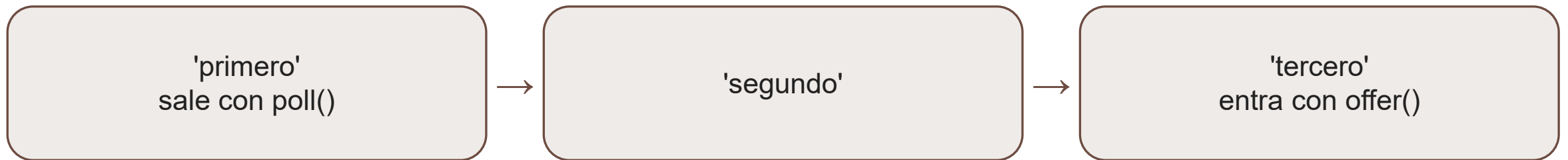
```
Deque<String> pila = new ArrayDeque<>();

pila.push("primero");
pila.push("segundo");
pila.push("tercero");

System.out.println(pila.pop()); // "tercero": el último en entrar sale primero
```

3.4 Colas (FIFO)

Una cola sigue la regla FIFO (First In, First Out): el primero en entrar es el primero en salir, como una fila de gente. LinkedList implementa la interfaz Queue.



Método	Qué hace
<code>offer(e)</code>	añade e al final de la cola
<code>poll()</code>	quita y devuelve el primer elemento
<code>peek()</code>	consulta el primer elemento, sin quitarlo

3.4 Colas — ejemplo

```
Queue<String> cola = new LinkedList<>();

cola.offer("primero");
cola.offer("segundo");
cola.offer("tercero");

System.out.println(cola.poll()); // "primero": el primero en entrar sale primero
```

No confundas push/pop (pila) con offer/poll (cola): las dos operan sobre un Deque o una Queue, pero sacan el elemento por un extremo distinto.

3.4 Conjuntos

Un Set guarda elementos sin admitir repetidos: si intentas añadir uno que ya está (según equals()), la segunda llamada a add() no hace nada.

	HashSet	LinkedHashSet	TreeSet
Orden de iteración	no garantizado	orden de inserción	ordenado según el valor

```
Set<String> nombres = new HashSet<>();

nombres.add("Ana");
nombres.add("Luis");
nombres.add("Ana"); // ya estaba: no se añade otra vez

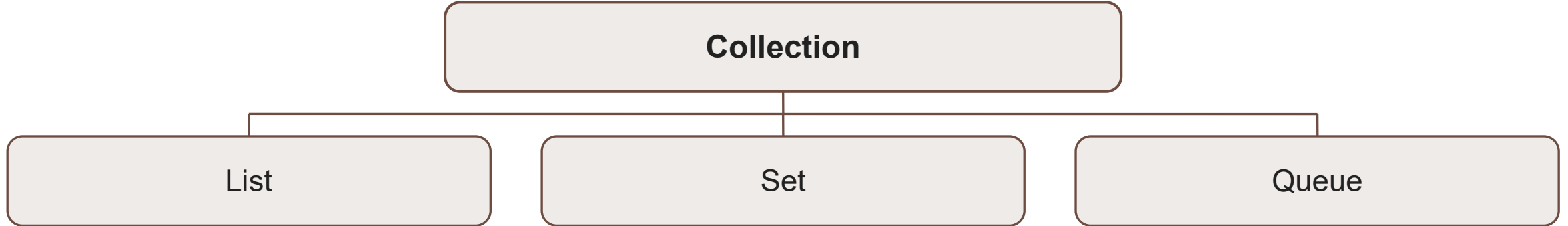
System.out.println(nombres.size()); // 2, no 3
```

3.4 Conjuntos — métodos

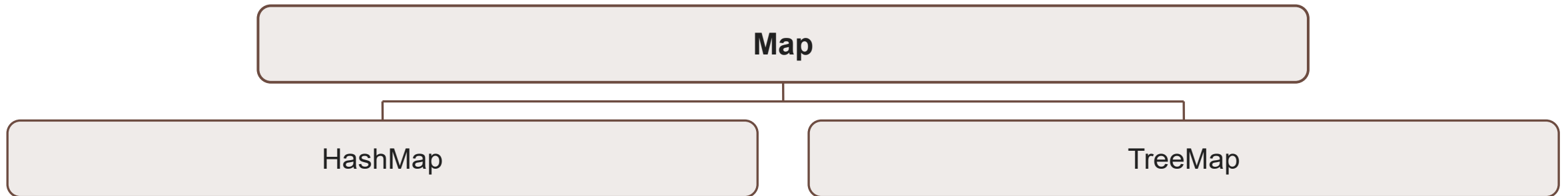
Método	Qué hace
<code>add(e)</code>	añade e, si no estaba ya
<code>remove(e)</code>	quita e, si estaba
<code>contains(e)</code>	comprueba si e está en el conjunto
<code>size()</code>	número de elementos

Por dentro, un `HashSet` es literalmente un `HashMap` donde tus elementos son las claves y el valor no se usa para nada. Por eso comparte con los mapas el mismo mecanismo de `equals()/hashCode()`.

3.5 Java Collection Framework



Independiente (no extiende de Collection):



Map no tiene `add()` ni se recorre con `for-each` directamente: hay que pasar antes por `keySet()`, `values()` o `entrySet()`.